

MYESCROW

# MyEscrow product and technical overview

Codebase snapshot reviewed: 2026-07-26

Status: functional staging MVP

# Purpose and scope

---

MyEscrow is a milestone-based transaction application for buyers and sellers. The current system includes a Next.js frontend, Fastify API, PostgreSQL/Prisma data model, retryable background worker, internal financial ledger, operations console, and staging deployment.

The word "escrow" currently describes an application-level ledger and simulated wallet. There is no payment processor, bank/custody integration, KYC/AML program, regulated funds flow, or production evidence-storage system. Treat all balances and top-ups as test data until those systems and the associated legal/compliance controls exist.

The implemented product model includes immutable agreement versions, idempotent money commands, an append-only escrow ledger, reconciliation, conditional state transitions, audit events, and recovery jobs. This document describes the product behavior and technical implementation as they exist in the reviewed snapshot.

# Codebase and runtime snapshot

| Item                     | Current value                                                  |
|--------------------------|----------------------------------------------------------------|
| Root repository          | <code>git@github.com:StefanGertz/myescrow.git</code>           |
| Root branch / commit     | <code>main / d5edf6a7d2c0aa080d7a838bb7bb95313c289641</code>   |
| Frontend repository      | <code>git@github.com:StefanGertz/MyEscrowFrontEnd.git</code>   |
| Frontend branch / commit | <code>master / fac67c48bda4db77c7b454f3258fc9e6d90969bb</code> |
| Frontend relationship    | Git submodule at <code>myescrow-web/</code>                    |
| API stack                | Node 20, TypeScript, Fastify 5, Prisma 5, PostgreSQL           |
| Web stack                | Next.js 16 App Router, React 19, React Query 5, Tailwind 4     |
| Email provider           | Resend                                                         |
| Frontend hosting         | Vercel                                                         |
| API hosting              | Oracle VM behind an Oracle load balancer                       |
| API image registry       | GitHub Container Registry                                      |
| Public frontend          | <code>https://app.myescrowdemo.xyz</code>                      |
| Public API               | <code>https://staging.myescrowdemo.xyz</code>                  |
| Operations UI            | <code>https://app.myescrowdemo.xyz/operations</code>           |

All three public endpoints returned HTTP 200 on 2026-07-26. That confirms reachability, not that the deployed build matches the Git commits above. The application has no public build/version endpoint, and the deployed SHA was not independently established.

# Product behavior

---

## Customer lifecycle

1. A person signs up as an individual or a business representative.
2. Email verification is required by default. Login returns an expiring JWT.
3. The creator chooses whether they are the buyer or seller, defines the counterparty, amount, description, and milestones, signs, and creates an escrow.
4. Creation atomically stores the escrow, agreement version, signature, invitation delivery, and outbox event. The email provider can fail without losing the escrow.
5. An existing counterparty receives the escrow immediately. A new or unverified counterparty claims pending escrows after signup/verification.
6. The counterparty can sign/approve, reject, or request agreement/milestone changes. Material changes create a new agreement version and invalidate earlier consent.
7. Both buyer and seller must sign the current locked agreement before funding.
8. The buyer funds from the internal MyEscrow wallet. Funding creates a ledger entry and debits the buyer's internal wallet.
9. The seller submits milestone work with a note and optional evidence metadata. Earlier milestones must be completed first.
10. The buyer can approve and release the milestone, request a revision with a reason, or open a dispute.
11. A dispute freezes only the affected held amount. Parties can add evidence metadata, propose a full seller/buyer allocation, accept the other party's proposal, or request arbitration after submitting evidence.
12. A funded cancellation can be mutual or unilateral. Mutual acceptance refunds eligible unreleased and undisputed funds. A unilateral request escalates without moving money.
13. Notifications, wallet history, escrow ledger history, and audit events provide a record of activity.

## Operational lifecycle

A separate worker:

- sends/retries invitation outbox messages;
- schedules invitation reminders and expiry;
- flags overdue funding;
- processes milestone review reminders and overdue escalation;
- sends dispute evidence reminders and closes/escalates evidence windows;
- escalates unanswered cancellation requests;
- reconciles escrow ledger, wallet transactions, and milestone state daily;
- records heartbeat and failure information.

Support/admin APIs expose worker health, failed jobs, outbox failures, aged escrows, duplicate command replays, reconciliation exceptions, dispute deadlines, arbitration requests, audit history, and safe retry/extension commands.

The web operations console implements health/alert views, failed operational-job retry, operator management, and escrow detail drill-down. Some support APIs do not yet have corresponding web controls; see "Known limitations and incomplete areas."

# System architecture

---

```
Browser
|
| same-origin /api requests
v
Next.js application on Vercel
|-- mock route handlers when NEXT_PUBLIC_USE MOCKS=true
`-- proxy route handlers when NEXT_PUBLIC_USE MOCKS=false
    |
    v
Fastify API on Oracle infrastructure
|
| Prisma
v
PostgreSQL
^
|
Operations worker ----> Resend email API
|
|-- outbox, deadlines, retries, reconciliation, heartbeat
```

Important boundaries:

- The browser normally calls Next.js route handlers under `/api/*`; those handlers either serve mocks or forward the request to Fastify. This avoids browser CORS issues.
- Authentication is a bearer JWT. The frontend keeps it in `sessionStorage`, restores it on refresh, and clears it at expiration or logout.
- Fastify is the domain and persistence boundary. Next.js route handlers should remain thin proxies/mocks.
- The API and worker use the same image and database but run as separate processes.
- PostgreSQL stores process state and financial records. Resend is used only for email delivery.

# Repository map

---

```
myescrow/
├── .github/workflows/backend-ci.yml
├── MYESCROW_TECHNICAL_OVERVIEW.md
├── docs/
│   ├── operations-incident-runbook.md
│   ├── unhappy-workflow-remediation-plan.md
│   ├── unhappy-vs-remediated-paths.md
│   └── unhappy-workflow-diagrams.md
├── index.html
├── myescrow-api/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   ├── seed.ts
│   │   └── migrations/
│   ├── scripts/
│   ├── src/
│   │   ├── config/
│   │   ├── plugins/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── tests/
│   │   ├── operationsWorker.ts
│   │   └── server.ts
│   ├── Dockerfile
│   ├── docker-compose.yml
│   └── docker-compose.staging.yml
├── myescrow-web/ # separate Git repository/submodule
│   ├── .github/workflows/ci.yml
│   ├── src/app/ # pages and Next.js API handlers
│   ├── src/components/
│   ├── src/hooks/
│   ├── src/lib/
│   └── src/tests/
```

The root `index.html` and generated documents are design/presentation artifacts, not runtime dependencies.

# Backend implementation

## Entry points

- `src/server.ts` : Fastify creation, CORS, plugins, error handling, route registration, health root.
- `src/routes/auth.ts` : signup, login, verification, password reset/change.
- `src/routes/dashboard.ts` : authenticated customer API.
- `src/routes/operations.ts` : authenticated support/admin API and customer audit route.
- `src/operationsWorker.ts` : continuous or one-shot recovery worker.
- `src/operatorCli.ts` : first-admin bootstrap.

## Service responsibilities

| Service                                | Responsibility                                                               |
|----------------------------------------|------------------------------------------------------------------------------|
| <code>dashboardService.ts</code>       | Dashboard read models and most escrow/agreement/milestone commands           |
| <code>agreementService.ts</code>       | Immutable agreement versions, signatures, locking, funding consent           |
| <code>moneyIntegrityService.ts</code>  | Internal wallet transfers, escrow ledger, balance derivation, reconciliation |
| <code>idempotencyService.ts</code>     | Request hashing, replay detection, original-response replay                  |
| <code>invitationService.ts</code>      | Transactional invitation outbox, delivery status, retry/backoff              |
| <code>milestoneReviewService.ts</code> | Submissions, review history, review deadlines                                |
| <code>disputeService.ts</code>         | Disputes, evidence metadata, settlements, arbitration request, cancellations |
| <code>operationsService.ts</code>      | Audit events, durable jobs, worker health, safe recovery commands            |
| <code>operatorService.ts</code>        | First admin, support/admin role management                                   |
| <code>emailService.ts</code>           | Resend payloads for verification, reset, invitation, and change requests     |
| <code>userService.ts</code>            | User lookup, password hashing/checking, account creation                     |

`dashboardService.ts` is 2,650 lines and `src/app/page.tsx` in the frontend is 5,042 lines. These are the two clearest refactoring targets.

## Data model

The authoritative schema is `myescrow-api/prisma/schema.prisma`. Main groups:

- Identity: `User`, `BusinessProfile`, `EmailVerificationToken`, `PasswordResetToken`
- Escrow: `Escrow`, `EscrowMilestone`
- Consent: `AgreementVersion`, `AgreementSignature`
- Delivery: `InvitationDelivery`, `OutboxEvent`

- Work review: `MilestoneSubmission`, `MilestoneEvidenceReference`, `MilestoneReview`
- Dispute/cancellation: `Dispute`, `DisputeEvidenceSubmission`, `DisputeResolutionAllocation`, `CancellationRequest`
- Money: `WalletTransaction`, `EscrowLedgerEntry`, `IdempotencyRecord`
- Operations: `OperationalJob`, `OperationalWorkerState`, `ReconciliationRun`, `AuditEvent`
- Read-model support: `Notification`, `TimelineEvent`, `Sequence`

Money is stored as integer cents and currently hard-coded to USD.

Many state fields are unrestricted `String` columns rather than database/application enums. Current escrow lifecycle values include:

```
pending_counterparty_signup
pending_approval
changes_requested
creator_signature_required
invitation_expired
funding_pending
funded
cancellation_pending
cancellation_review
dispute_resolution_pending
completed
cancelled
rejected
```

Milestone values include `not_started`, `submitted`, `revision_requested`, `released`, `disputed`, `settled`, and `cancelled`. Dispute values include `open`, `resolution_proposed`, `arbitration_requested`, `resolving`, and `resolved`.

Because these are strings spread across services, search all transitions before changing a status name.

## Financial model and invariants

The internal wallet is a database balance on `User`; top-up and withdrawal endpoints directly change it. No external money moves.

Escrow money movements are append-only `EscrowLedgerEntry` rows with linked wallet transactions:

- `fund`
- `release`
- `refund`
- `settlement_release`
- `settlement_refund`

The core invariant is:

```
funded = held + released + refunded
held >= 0
released + refunded <= funded
```

Active disputes are subtracted from available held funds. `applyEscrowTransfer` performs the process update, internal wallet movement, ledger record, and invariant check inside a Prisma transaction. Reconciliation compares ledger totals, milestones, and linked wallet transactions.

The legacy full-escrow release endpoint remains registered for compatibility but deliberately throws an error. Milestone approval is the supported release path.

## Idempotency

Consequential financial and workflow commands require an `Idempotency-Key` header of 8–200 characters. The record is unique by user and key. A matching replay returns the original JSON response; reuse with another command/payload returns HTTP 409.

The strongest coverage is on:

- escrow creation;
- funding;
- milestone submit/approve;
- dispute open/evidence/proposal/arbitration/resolve;
- funded cancellation request/accept;
- wallet top-up/withdrawal;
- operational retry/extension/role changes.

Not every non-financial mutation uses the idempotency service. Sending the header alone does not make a route idempotent; the called service must use `idempotencyService`.

## API surface

All dashboard and operations routes require bearer authentication except auth endpoints and `GET /`. The route files are the source of truth.

Main route groups:

| Group                | Examples                                                                                    |
|----------------------|---------------------------------------------------------------------------------------------|
| Auth                 | signup, login, verify/resend email, forgot/reset/change password                            |
| Dashboard reads      | overview, escrows, business profile, disputes, notifications, wallet history                |
| Agreement            | create/update escrow, approve/reject, sign, request/apply changes, invitation resend/extend |
| Money                | fund, milestone approve, ledger history, top-up, withdraw                                   |
| Work review          | submit/resubmit milestone, request revision, apply milestone changes                        |
| Dispute/cancellation | open dispute, evidence, proposal, arbitration, resolve, request/accept cancellation         |
| Operations           | health, jobs, retry, audit, evidence, invitation recovery, operator roles                   |

The API README route table is useful but is not exhaustive relative to the registered routes.

## Worker behavior

Run continuously:

```
npm run operations:run
```

Run one cycle:

```
npm run operations:once
```

The staging Compose file runs the continuous worker every minute. It stores jobs, claims them conditionally, retries with backoff, recovers locks older than ten minutes, and updates the `primary` worker heartbeat. A healthy deployment should show a successful heartbeat within two minutes.

See [docs/operations-incident-runbook.md](#) before manually intervening. Do not repair ledger, job, outbox, or operator state by editing PostgreSQL.

# Frontend implementation

## Main application modes

The same codebase contains three meaningful combinations:

| NEXT_PUBLIC_USE MOCKS | NEXT_PUBLIC_LIVE_DASHBOARD | Result                                           |
|-----------------------|----------------------------|--------------------------------------------------|
| true or unset         | false or unset             | Immersive demo UI with Next.js mock API handlers |
| false                 | false or unset             | Immersive UI with live Fastify data              |
| false                 | true                       | Smaller LiveDashboard with live Fastify data     |

`src/app/page.tsx` is the immersive application and contains most screens and transaction workflows in one file.  
`src/components/LiveDashboard.tsx` is the smaller live layout.

## API flow

- React Query hooks live in `src/hooks/useAuthApi.ts` and `src/hooks/useDashboardData.ts`.
- `src/lib/apiClient.ts` adds the runtime bearer token.
- Browser requests to `/api/*` remain same-origin.
- Every Next.js route handler either returns mock data or uses `src/lib/serverProxy.ts` to forward method, headers, and body to Fastify.
- Mock fixtures and many shared response types live in `src/lib/mockDashboard.ts`.

`NEXT_PUBLIC_API_BASE_URL` is read by server route code but is named as a public browser variable, so its value is part of public build-time configuration.

## Authentication

- `AuthProvider` owns the current user/token.
- Sessions are kept in `sessionStorage`, not durable `localStorage`.
- The token is cleared on expiration, logout, or browser-session end.
- Default API JWT lifetime is eight hours (`AUTH_SESSION_TTL_SECONDS`).
- There are no refresh tokens, server-side revocation list, device/session management, or forced invalidation after password change.
- Customer and operations login use the same underlying JWT/account system; authorization is enforced by API role checks.

## Mock credentials

Mock mode accepts:

Email: `scott@example.com`  
Password: `Escrow123!`

The Prisma seed creates verified `scott@example.com` and `nora@example.com` records from `data/store.json`. This overview does not include a plaintext live/seed password; local access can be established through the reset flow or fresh accounts.

# Local setup

---

## Clone correctly

The frontend is a submodule:

```
git clone --recurse-submodules git@github.com:StefanGertz/myescrow.git
cd myescrow
git submodule update --init --recursive
```

Commit frontend work in `myescrow-web` first, push it, then commit the updated submodule pointer in the root repo.

## Prerequisites

- Node.js 20+
- npm 10+
- Docker Desktop or another reachable PostgreSQL instance
- Git/SSH access to both repositories

Versions used for this review:

```
Node v20.20.2
npm 10.8.2
Docker 29.6.1
```

## API

```
cd myescrow-api
npm ci
cp .env.example .env
docker compose up -d db
npm run db:migrate
npm run db:seed
npm run dev
```

The API defaults to `http://localhost:4000`.

Important: `npm run db:seed` deletes and recreates application data in the configured database. Verify `DATABASE_URL` before running it. Never seed staging or production.

Recommended local `.env`:

```
PORT=4000
JWT_SECRET=replace-with-a-long-random-local-secret
AUTH_SESSION_TTL_SECONDS=28800
DATABASE_URL=postgresql://myescrow:myescrow@localhost:5432/myescrow
AUTH_REQUIRE_EMAIL_VERIFICATION=true
AUTH_DEBUG_CODES=true
EMAIL_VERIFICATION_CODE_DIGITS=6
EMAIL_VERIFICATION_TTL_MINUTES=15
PASSWORD_RESET_CODE_DIGITS=6
PASSWORD_RESET_TTL_MINUTES=15
APP_URL=http://localhost:3000
EMAIL_FROM="MyEscrow <hello@myescrow.test>"
EMAIL_REPLY_TO=""
RESEND_API_KEY=
```

With no Resend key, verification/reset codes are logged locally, but escrow invitation delivery enters a visible failed/retryable state.

## Frontend against mocks

```
cd myescrow-web
npm ci
printf 'NEXT_PUBLIC_USE MOCKS=true\nNEXT_PUBLIC_LIVE_DASHBOARD=false\n' > .env.local
npm run dev
```

## Frontend against the local API

Create `myescrow-web/.env.local`:

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:4000
NEXT_PUBLIC_USE MOCKS=false
NEXT_PUBLIC_LIVE_DASHBOARD=false
```

Then:

```
cd myescrow-web
npm run dev
```

The frontend defaults to `http://localhost:3000`.

Do not use `NEXT_PUBLIC_API_TOKEN` in a shared or deployed environment. It is a build-exposed fallback intended for temporary development.

# Quality gates

---

Verified on the exact commits in this document:

| Project  | Command                    | Result                            |
|----------|----------------------------|-----------------------------------|
| API      | <code>npm test</code>      | 1 file, 36 tests passed           |
| API      | <code>npm run build</code> | passed                            |
| API      | <code>npm run lint</code>  | TypeScript no-emit check passed   |
| Frontend | <code>npm test</code>      | 11 files, 38 tests passed         |
| Frontend | <code>npm run build</code> | passed; 32 pages/routes generated |
| Frontend | <code>npm run lint</code>  | passed                            |

Backend tests create an isolated PostgreSQL schema, deploy all 17 migrations, seed it, run the suite, and drop the schema. If no test/database URL is reachable, the runner can start a disposable PostgreSQL 16 container.

There is no measured coverage threshold, browser end-to-end suite, payment-provider contract suite, load test, penetration test, accessibility audit, or migration rollback test.

CI:

- Root `.github/workflows/backend-ci.yml` runs migrations, seed, backend tests, build, a smoke test, and docs encoding check. On `main`, it publishes mutable `latest`, `v1`, and immutable commit-SHA GHCR tags.
- Frontend `.github/workflows/ci.yml` runs lint, tests, and build on `master / main` and pull requests.
- Vercel deployment is triggered from the frontend repository, while API image publication is triggered from the root repository. A root submodule update does not by itself replace the frontend deployment.

# Staging and deployment

---

Current documented topology:

```
app.myscrowdemo.xyz
  -> Vercel project serving myescrow-web

staging.myscrowdemo.xyz
  -> Oracle load balancer
  -> Oracle VM
  -> Docker Compose:
      PostgreSQL
      Fastify API
      operations worker
```

The API image is `ghcr.io/stefangertz/myescrow-api`. The live host uses `myescrow-api/docker-compose.staging.yml` and a host-only `.env.staging`.

The ignored `myescrow-api/.env.staging` on this workstation is incomplete and is not the staging source of truth.

## Deployment mechanics

The deployment flow represented by the repository is:

1. Backend CI tests and builds the API.
2. A successful `main` build publishes GHCR tags for `latest`, `v1`, and the commit SHA.
3. Prisma migrations are applied separately with `npm run db:migrate:deploy`.
4. The API and operations worker are updated from the same image.
5. Health, worker heartbeat, reconciliation state, and the smoke flow are checked after deployment.

The current helper script and Compose flow do not form a complete migration/rollback release system.

`scripts/deploy-staging.sh` writes `.env.staging` and starts the services, but does not run Prisma migrations.

The API also has no version endpoint that identifies the deployed source SHA.

# Known limitations and incomplete areas

---

## Funds, evidence, and dispute authority

- Wallet top-up and withdrawal update database balances only. No payment provider, bank, custody account, webhook flow, settlement state, reversal, or chargeback integration exists.
- KYC/KYB, AML/sanctions screening, transaction monitoring, regulated custody, and jurisdiction-specific compliance are outside the current system.
- Evidence models contain object keys and file metadata, but there is no upload/download service, private object-store authorization, malware scanning, retention workflow, or encryption/key policy.
- Arbitration creates an `arbitration_requested` state and operations alert. It does not implement a staff adjudication or staff-authorized payout command.
- Currency is effectively fixed to USD even though currency appears on ledger and agreement records.

## Security and authentication

- `JWT_SECRET` falls back to `dev-secret-change-me` when absent, including under a production environment.
- Verification and password-reset codes are written to application logs by `emailService.ts`.
- Login, signup, verification, resend, forgot/reset password, invitation, and financial endpoints have no rate limits, account lockout, or bot/abuse controls.
- Fastify CORS is configured with `origin: true`.
- Browser authentication uses a bearer JWT in `sessionStorage`. There are no refresh tokens, server-side revocation, device/session management, or operator MFA.
- Password changes do not revoke already issued JWTs.
- No penetration test, dependency security gate, formal threat model, or authorization-matrix suite is represented in the repository.

## Functional and client/API inconsistencies

- `LiveDashboard` submits escrow creation without the PNG signature required by the Fastify schema. The frontend hook marks `signatureDataUrl` optional, so the mismatch is not caught by TypeScript.
- Frontend mutation hooks create a new UUID when a mutation is invoked. A separate user retry after an uncertain response uses a new idempotency key rather than the original command key.
- The operations API supports outbox retry, invitation extension, dispute evidence inspection, and audit history, while the web operations interface exposes only part of that functionality.
- The immersive dashboard and `LiveDashboard` are different product surfaces with different feature coverage.
- Some Next.js mock handlers implement behavior that is intentionally simpler than the live API.
- The API README route table is incomplete relative to registered Fastify routes.

## Code structure, testing, and operations

- `dashboardService.ts` is 2,650 lines and owns both read models and many commands. The immersive `src/app/page.tsx` is 5,042 lines.
- Frontend response types are partly derived from mock models rather than a shared/generated API contract.
- Lifecycle, milestone, delivery, and dispute statuses are unrestricted strings spread across services.

- Automated coverage consists primarily of API integration tests and frontend component/utility tests. There is no browser end-to-end suite, coverage threshold, load test, accessibility audit, payment-provider contract suite, or migration rollback test.
- Dashboard freshness relies partly on polling.
- Observability is limited to structured application/container logs, stored worker heartbeat, operations health queries, and database reconciliation records; there is no tracing or error-tracking integration.
- Local and test Compose use PostgreSQL 16, while staging Compose declares PostgreSQL 15.
- `.env.example` contains the unused `DATA_STORE_PATH` variable.
- `docs/unhappy-workflow-diagrams.md` describes the earlier unsafe workflow, while the remediation documents describe later implementation phases.

# Product behavior not yet defined

---

The code makes temporary assumptions in areas where final product behavior has not been defined:

- Supported countries, currencies, transaction sizes, prohibited activities, and user types
- Whether MyEscrow is a custodian, marketplace, agent, or UI over a licensed provider
- Invitation, funding, review, evidence, cancellation, and dispute deadlines
- Buyer non-response policy (current behavior is hold and escalate, never auto-release)
- When milestones may proceed out of order
- Who can cancel at each lifecycle state
- Who adjudicates arbitration and how allocations are authorized
- Refund, reversal, chargeback, fee, tax, and FX behavior
- Evidence/file retention and privacy rights
- Notification channels and legally effective notice
- Support service levels and escalation ownership

# Useful commands

---

API:

```
cd myescrow-api
npm run dev
npm test
npm run lint
npm run build
npm run smoke
npm run db:migrate
npm run db:migrate:deploy
npm run db:seed
npm run reconcile:ledger
npm run operations:dev
npm run operations:run
```

Frontend:

```
cd myescrow-web
npm run dev
npm test
npm run lint
npm run build
```

Repository/submodule:

```
git status --short
git submodule status
git -C myescrow-web status --short
git -C myescrow-web log -1 --oneline
```

## Related technical documents

---

- [myescrow-api/README.md](#) : API setup, scripts, route summary, testing, and deployment notes
- [myescrow-web/README.md](#) : frontend setup, runtime modes, proxy behavior, and authentication
- [docs/operations-incident-runbook.md](#) : worker behavior, support roles, and incident procedures
- [docs/unhappy-workflow-remediation-plan.md](#) : rationale behind the ledger, agreement, evidence, dispute, and recovery models
- [docs/unhappy-vs-remediated-paths.md](#) : comparison of the earlier and current failure-handling paths